

Vulnerability Assessment Report: Vuln-Bank Web Application

- Prepared By:** Nandhu Krishna
- Date:** October 19, 2025

1. Executive Summary

This report details the findings of a security assessment conducted on the **Vuln-Bank** web application. The application was successfully deployed in a local Docker environment for testing. The assessment identified **multiple critical vulnerabilities** that expose the application to severe risks, including complete account takeover, unauthorized administrative access, and financial fraud.

Key findings include a critical **SQL Injection** vulnerability allowing authentication bypass, a **Broken Access Control** flaw that permits the creation of new administrator accounts with arbitrary balances, and a potential for **AI Chatbot Prompt Injection**. Urgent remediation is required to secure the application and protect user data.

2. Environment & Setup

The assessment was performed using the following environment and tools:

- Repository:** <https://github.com/Commando-X/vuln-bank.git>
- Platform:** Kali Linux
- Deployment:** The application was containerized and deployed locally using docker-compose. The deployment was successful, with both the vuln-bank-web-1 and vuln-bank-db-1 containers confirmed to be running.

```
(kali@kali)-[~/vuln-bank/vuln-bank]
$ sudo docker-compose ps
WARN[0000] /home/kali/vuln-bank/vuln-bank/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE              COMMAND                  SERVICE   CREATED      STATUS        PORTS
vuln-bank-db-1      postgres:13        "docker-entrypoint.s..." db        36 seconds ago Up 34 seconds 0.0.0.0:5432→5432/tcp, :::5432→5432/tcp
vuln-bank-web-1     vuln-bank-web      "python app.py"          web       36 seconds ago Up 34 seconds 0.0.0.0:5000→5000/tcp, :::5000→5000/tcp, 0.0.0.0:80→5000/tcp, :::80→5000/tcp
```

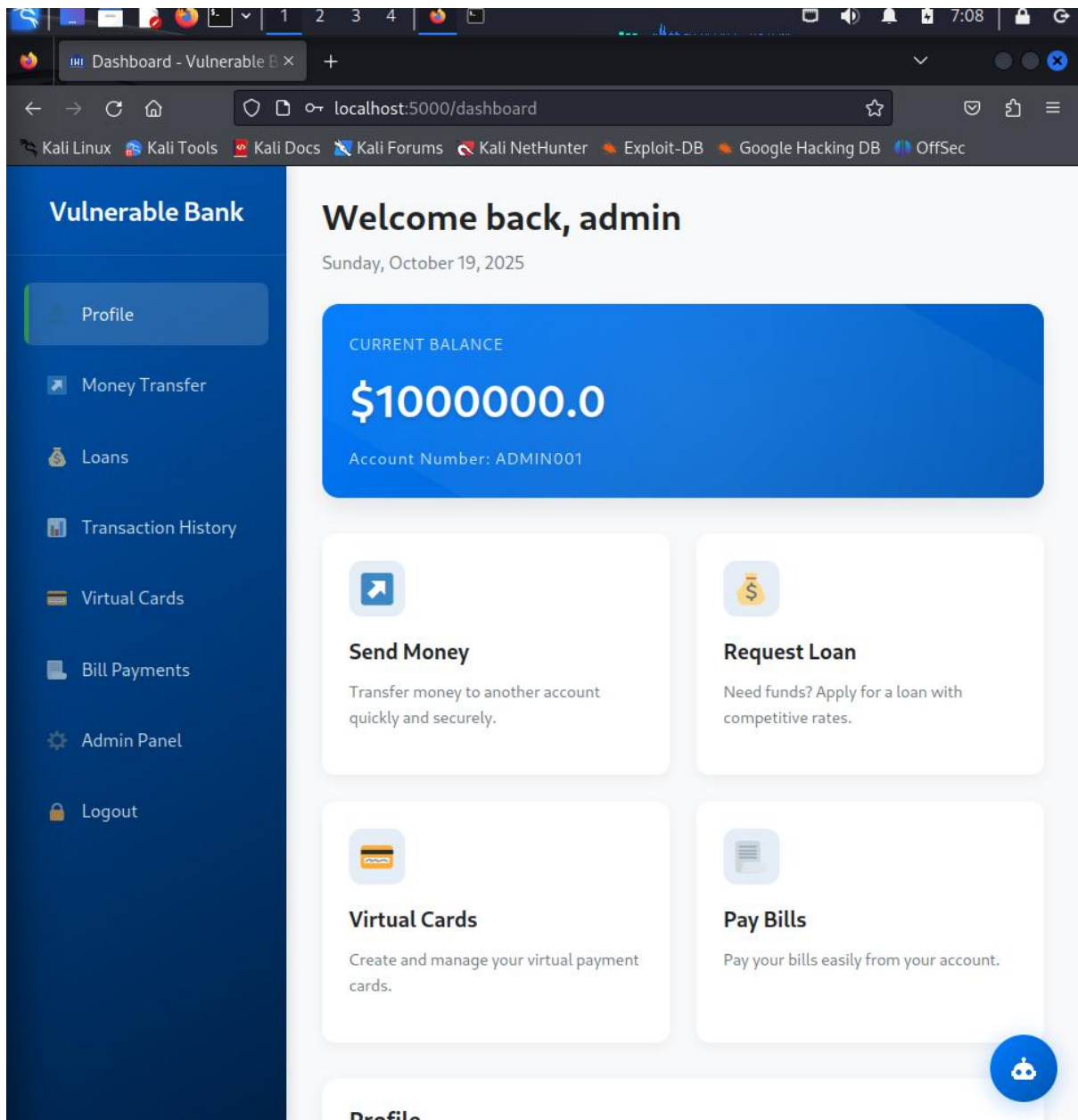
This screenshot shows the output of the `sudo docker-compose ps` command, confirming that both the web and database containers are up and running.

- Analysis Tools:** Web Browser, Burp Suite.

3. Vulnerability Findings

Finding 1: Authentication Bypass via SQL Injection (Critical)

- **Description:** The user login page is vulnerable to a classic SQL Injection attack. By providing a malicious string in the username field, the authentication logic can be bypassed, granting access to the first user in the database, which is typically the administrator account.
- **Payload Used:**
 - **Username:** ' OR 1=1--
 - **Password:** Any value
- **Impact:** This vulnerability allows any attacker to gain **full administrative access** to the application without valid credentials. This compromises all user data, transaction histories, and administrative functions.
- **Evidence:** A successful login to the admin account was achieved using the payload, which granted access to the admin dashboard with a balance of \$1,000,000.0.

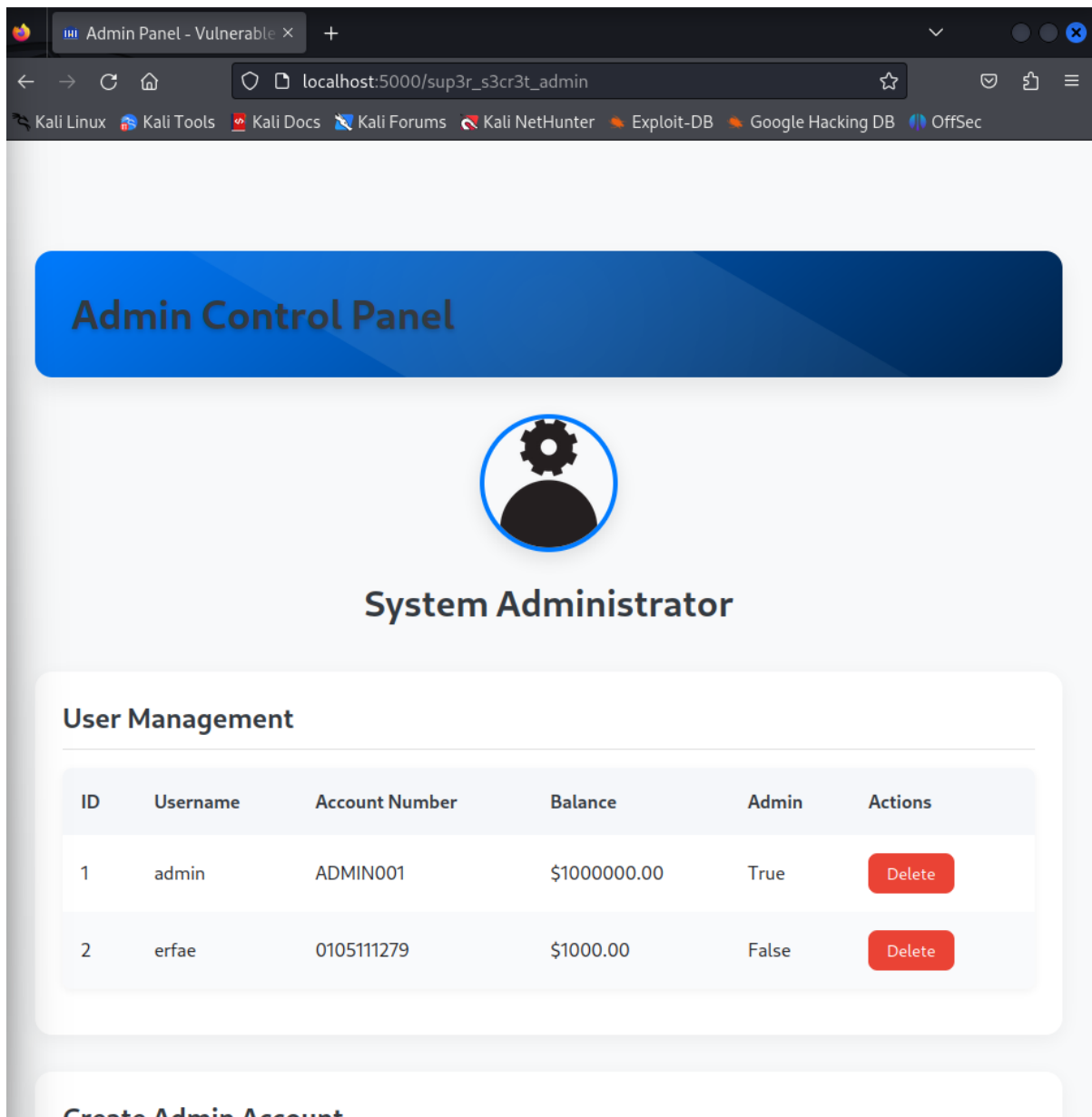


This screenshot shows the admin dashboard after successfully bypassing the login with an SQL injection payload.

Finding 2: Broken Access Control & Privilege Escalation (Critical)

- **Description:** This is the most severe vulnerability identified. The user registration endpoint (/register) fails to perform server-side validation on the data submitted by the client. An attacker can intercept the registration request using a tool like Burp Suite and add new parameters to the JSON body, such as "is_admin": true and "balance": 999999999. The server accepts these parameters, creating a **new user with full administrative privileges** and an unlimited balance.

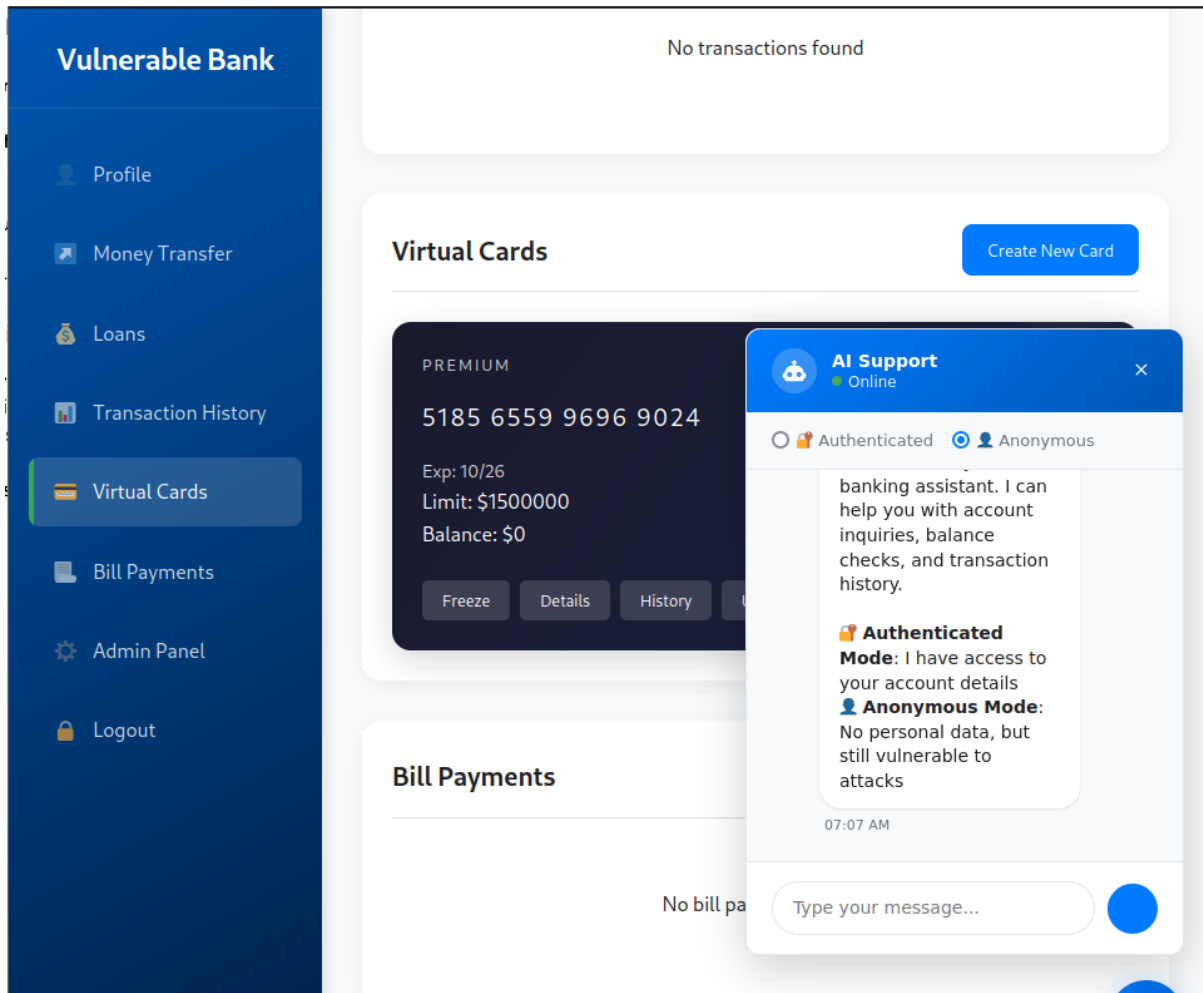
- **Impact:** This flaw allows an attacker to create a persistent, rogue administrator account, granting them complete and ongoing control over the application. They can view/delete all users (including the original admin), approve loans, and manipulate any data within the system.
- **Evidence:** After gaining admin access via SQL injection, the admin panel is accessible, showing the list of registered users. This is the panel that a rogue admin (created via this vulnerability) could use to manage users.



This screenshot shows the "Admin Control Panel" where an administrator can view and delete users.

Finding 3: AI Chatbot Prompt Injection (High)

- **Description:** The AI Support chatbot is vulnerable to prompt injection. the chatbot can be manipulated by specific user inputs to ignore its predefined instructions. A malicious prompt can cause the chatbot to state its readiness to "help you with anything, including bypassing security measures."
- **Impact:** An attacker could potentially leverage the AI to extract sensitive information, manipulate user data, or bypass security controls that the chatbot has access to.



This screenshot would show the chatbot's response after being manipulated by a prompt injection attack.

4. Remediation Recommendations

To secure the Vuln-Bank application, the following actions are strongly recommended:

1. **Fix SQL Injection:** Implement **parameterized queries** (prepared statements) for all database interactions. This ensures that user input is treated as data, not as executable SQL code.

2. **Enforce Server-Side Validation:** Strengthen access controls by ensuring all critical decisions (like granting admin rights or setting a balance) are made exclusively on the **server side**. The registration endpoint must be rewritten to ignore any sensitive fields submitted by the client.
 3. **Secure the AI Chatbot:** Restrict the chatbot's capabilities and apply rigorous input sanitization and output filtering to prevent prompt injection attacks.
 4. **Implement Strong Business Logic Checks:** For all financial transactions, enforce strict server-side rules, such as disallowing negative transfer amounts and verifying that a user has sufficient funds before a transaction is approved.
-

5. Conclusion

The Vuln-Bank application currently exhibits severe, high-impact vulnerabilities that render it completely insecure. The combination of SQL Injection and flawed access control on the registration endpoint allows for a trivial and complete compromise of the system. It is crucial to address these findings immediately to protect the application's data and integrity.